# The "Hybrid Digital Twin"

**The Goal:** Synchronize 3D Gaussian Splatting (3DGS) with functional CAD geometry.

## 1. Detailed Requirements

### Phase 1: Hybrid Rendering Pipeline

- Integrate a 3D Gaussian Splatting renderer (e.g., .splat format) into a standard Three.js scene.
- Load a high-poly mechanical .stl or .glb mesh (e.g., a car engine or gearbox) into the center of the splat environment.

### Phase 2: Depth-Buffer Synchronization

- **The Problem:** Standard 3DGS renderers use alpha-sorting that often breaks the WebGL depth buffer, causing meshes to "bleed through" splats.
- **Task:** Modify the rendering loop or shaders to ensure the CAD mesh is correctly occluded by splats that are physically in front of it.
- **Requirement:** The mesh must receive and cast shadows within the hybrid environment (if the splat data permits a floor plane).

### Phase 3: Proximity & Logic (Senior)

- **Custom Shader:** Write a ShaderMaterial or use onBeforeCompile for the CAD mesh.
- **Logic:** As the user moves the mesh closer to any "dense" area of the splat cloud (the floor or walls), the mesh surface must dynamically change color or glow. Pass the distance data via a Uniform.
- **State Persistence:** Implement a SceneStateSerializer to save/load the mesh position, rotation, and scale as a JSON string in localStorage.

# Submission & Evaluation Criteria

**How to Submit**

1. **Source Code:** Provide access to a private GitHub repository or a Zip file.
2. **README.md:** Must include:
    - Instructions to run (e.g., npm install && npm run dev).
    - A section titled **"Architectural Decisions"** explaining why you chose your specific spatial indexing or depth-sync strategy.
    - A section titled **"Bottlenecks Handled"** describing how you managed memory and UI performance.